THE TARDIS PLASMA MODULE

Plasma Structure

NetworkX - Plasma Graphs

The plasma is structured as a network of calculations. Each quantity (temperature, ionization, optical depth, etc.) is a "property." Properties depend on one another, and TARDIS connects them into a graph so that when one value changes, only the downstream values get recalculated, in the right order. The plasma built in Tardis simulations lives in the class BasePlasma, which is defined in plasma/base.py. Here's a breakdown of how the plasma is structured as a graph with the NetworkX package.

First, every property declares what it's going to output, and there are two types of properties:

Input properties: values fed in from the model or config. They have outputs but no inputs, so they are the starting points of the graph.

Processing properties: values that get calculated. TARDIS reads the argument names of their calculated function to learn what they need.

For example, the property PhiSahaLTE (defined in plasma/properties/ion_population.py) has the function **calculate(g_electron, beta_rad, partition_function, ionization_data)** and produces the output phi. So its inputs are those four names, and its output is phi.

Connections are made by matching names. An output called phi automatically links to any calculation that has an argument called phi.

Next is actually building the graph in the **_build_graph** function. Tardis builds the graph in three steps:

1. Add one node for every property.
2. Mark the input properties as the starting points (they have no inputs).
3. For each calculated property, look at what it needs, find the property that produces each of those names, and draw an arrow from producer to consumer.

If a property needs something that nothing produces, TARDIS stops with an error. This catches broken or incomplete setups immediately.

When an input changes (say, the temperature), Tardis finds everything downstream of the change and recalculates those in topological order, so each property is updated only after the values it depends on are fresh.

The following documentation shows how to display a plasma graph:

https://tardis-sn.github.io/tardis/how_to/output/how_to_plasma_graph.html

Plasma Solver Factory

The PlasmaSolverFactory class defined in plasma/assembly/base.py initializes, configures, and assembles the plasma used in tardis runs. The factory has a prepare_factory method, which gets passed specific property collections that are collections of classes that define specific plasma properties. The plasma properties are described in further detail later in the document. The assemble method in the factory returns an instance of the BasePlasma class.

There is also an assemble_plasma function defined in two locations: plasma/assembly/legacy_assembly.py and plasma/standard_plasmas.py. The assemble_plasma function used by the simulation class in Tardis is the one defined in plasma/assembly. The function returns an assembled plasma by defining a PlasmaSolverFactory instance, calling the class's prepare_factory method, and finally calling the class's assemble method.

Plasma Properties

The plasma properties used throughout the plasma module are in a file called legacy_property_collections.py. There is another file called property_collections.py which has less classes collected in the file. The plasma that is actually built throughout the Tardis codebase imports from the legacy_property_collections.py file, and property_collections.py is a file that is not referenced in other parts of the Tardis codebase for plasma that actually gets built by tardis runs.

Here is a report on what classes make up the properties and whether these classes are ever actually built in the plasma module or referenced outside of their class definitions. These are the property categories defined in legacy_property_collections.py. A later section details the property collection differences for legacy_property_collections.py and property_collections.py

Green Highlight - Covered in integration tests with unit tests written if unit tests were deemed useful
Yellow Highlight - Referenced in Tardis codebase but never run in integration tests. This indicates that this class is not usually built when real tardis users run tardis

Red Highlight - Never referenced outside class definition

Unhighlighted - Will have a note next to it explaining why it can't fit into the highlight categories.

Adiabatic Cooling Properties

- AdiabaticCoolingRate

Basic Inputs

There is no need to run unit tests for Inputs, in this case the green highlight indicates that this specific input is used for classes that are built by plasma that tardis users actually use, the red highlight indicates that the input is usually never used.

- DilutePlanckianRadField
- NumberDensity
- TimeExplosion
- AtomicData
- JBlues
- LinkTRadTElectron
- HeliumTreatment
- ContinuumInteractionSpecies - Never used as an input in any integration test and many of the properties/classes associated with continuum interaction species seem to be part of an unfinished refactor attempt
- NLTEIonizationSpecies
- NLTEExcitationSpecies

Basic Properties

- BetaRadiation
- Dilution Factor
- ElectronTemperature
- GElectron
- IonizationData
- Levels
- Lines
- LinesLowerLevelIndex
- LinesUpperLevelIndex
- PartitionFunction
- SelectedAtoms
- StimulatedEmission
- TRadiative
- TauSobolev

Continuum Interaction Inputs

- PhotoIonRateCoeff
- StimRecombRateFactor

- BfHeatingRateCoeffEstimator
- StimRecombCoolingRateCoeffEstimator
- YgData - referenced in plasma/equilibrium/tests/test_collisional_transitions.py and plasma/tests/test_plasma_continuum.py but not referenced in the plasma solver factory

Continuum Interaction Properties

- BoundFreeOpacity
- CollDeexcRateCoeff - Imported into collision_strengths.py and is covered when the rate solver code is tested in plasma/equilibrium/tests/test_collisional_transitions.py. This class is defined in two files: plasma/equilibrium/rates/collision_strengths.py and plasma/properties/continuum_processes/rates.py. The class imported into the properties collection is the class defined in rates.py, and the class imported into test_collisional_transitions is also from rates.py which means the class defined in plasma/equilibrium/rates/collision_strengths.py is fully uncovered
- CollExcRateCoeff - same as above
- CollIonRateCoeffSeaton
- CollRecombRateCoeff
- ContinuumInteractionHandler
- CorrPhotoIonRateCoeff
- FreeBoundCoolingRate
- FreeBoundEmissionCDF
- FreeFreeCoolingRate
- LevelIdxs2LineIdx
- LevelIdxs2TransitionIdx
- LevelNumberDensityLTE
- MarkovChainIndex
- MarkovChainTransProbs
- MarkovChainTransProbsCollector
- MonteCarloTransProbs
- NonContinuumTransProbsMask
- NonMarkovChainTransitionProbabilities
- PhotoIonBoltzmannFactor
- PhotoIonizationData - Referenced in the main plasma solver factory but never run, this is a class that also exists for the iip plasma
- RawCollIonTransProbs
- RawCollisionTransProbs
- RawPhotoIonTransProbs
- RawRadBoundBoundTransProbs
- RawRecombTransProbs
- SahaFactor
- SpontRecombCoolingRateCoeff
- SpontRecombRateCoeff - Only defined in the class and put into property collections but never referenced in the main plasma solver factory. This class also exists for the iip plasma.

- StimRecombRateCoeff
- ThermalGElectron
- ThermalLTEPartitionFunction
- ThermalLevelBoltzmannFactorLTE
- ThermalPhiSahaLTE
- YgInterpolator
- AdiabaticCoolingRate

Dilute LTE Excitation Properties

- LevelBoltzmannFactorDiluteLTE

Helium LTE Properties

- LevelNumberDensity
- IonNumberDensity

Helium NLTE Properties

- HeliumNLTE
- RadiationFieldCorrection
- ZetaData
- BetaElectron
- LevelNumberDensityHeNLTE
- IonNumberDensityHeNLTE

Helium Numerical NLTE Properties

- HeliumNumericalNLTE - This particular property requires a specific numerical NLTE solver and a specific atomic dataset (neither of which are distributed with Tardis) to work.

LTE Excitation Properties

- LevelBoltzmannFactorLTE

LTE Ionization Properties

- PhiSahaLTE

Nebular Ionization Properties

- PhiSahaNebular
- ZetaData
- BetaElectron

- RadiationFieldCorrection

NLTE lu Solver Properties

- NLTEIndexHelper
- NLTEPopulationSolverLU - referenced in test_nlte_solver but never run in full integration test.

NLTE Properties

- LevelBoltzmannFactorNLTE
- NLTEData
- PreviousElectronDensities
- PreviousBetaSobole
- BetaSobolev

NLTE Root Solver Properties

- NLTEIndexHelper
- NLTEPopulationSolverRoot - referenced in test_nlte_solver but never run in full integration test.

Non NLTE Properties

- LevelBoltzmannFactorNoNLTE

Two Photon Properties

- RawTwoPhotonTransProbs
- TwoPhotonData
- TwoPhotonEmissionCDF
- TwoPhotonFrequencySampler

Equillibrium

In the tardis/plasma/equilibrium folder there are files under a rates folder and seem to be another attempt at a plasma module refactor that uses “RateSolver”s and are not used in the actual implemented plasma module used in simulations. All of these solvers do have direct tests though.

Uncovered Plasma Module Files

These are files in the plasma module that are either fully uncovered or have very little coverage and was not already noted by documenting the uncovered classes in the properties

- tardis/plasma/standard_plasmas.py

    - 0% coverage, seems to be an attempt at refactoring how the assemble_plasma function is used by the PlasmaSolverFactory. It gets referenced nowhere else in the codebase except for a notebook called grotarian_mockup.ipynb
- tardis/plasma/properties/property_collections.py
    - As mentioned earlier, this is the property collection for a plasma module refactor that was never completed. It's seems to be imported into tardis/plasma/standard_plasmas.py
- tardis/plasma/properties/nlte_excitation_data.py
    - Defines NLTEExcitationData which is used anywhere else in the codebase
- tardis/plasma/properties/hydrogen_continuum.py
    - 0% coverage by plasma tests
- tardis/plasma/properties/continuum_processes/fast_array_util.py
    - Defines two numba functions for integration
        - numba_cumulative_trapezoid
            - Used in an uncovered class TwoPhotonEmissionCDF in tardis/plasma/properties/continuum_processes/rates.py
        - cumulative_integrate_array_by_blocks
            - Same as above function

Tools to Understand Plasma Module

Test Coverage Reports

Generating coverage reports of the plasma module test cases is an incredibly useful way to understand what parts of the plasma module are actually used by researchers using Tardis. Here's an example of how I used the VSCode test coverage tool when writing up this documentation:

After running exclusively the tests in the plasma module I noticed a file that is fully uncovered in test coverage - plasma/standard_plasmas.py. In this file there is a function called assemble_plasma. This is never used in the tardis simulation that actually creates and updates plasma and is therefore never used in tardis. The assemble_plasma that is used in tardis is in plasma/assembly/legacy_assembly.py.

In the PlasmaSolverFactory class in plasma/assembly/base.py there is this line of code

The red highlight shows that the continumm_interactions are never set up because none of the plasma configs tested ever set up continuum interactions. It seems that continuum interactions are never set up in type i supernovae and should be removed from the type i plasma completely or a plan needs to be put into place to make continuum interactions work with type i supernovae.

hydrogen_continuum.py is not used anywhere and has 0% coverage

The best way to find which classes are actually getting called is to go through the legacy properties collection, click on the classes, and see which classes actually have the code in the calculate function actually run.

This LinesLowerLevelIndex class is in legacy properties and has full coverage, which means when the test_full_plasmas class is building 20 different plasmas this plasma property is actually getting calculated and used, rather than just being imported from this properties collection.

```python
class LinesLowerLevelIndex(HiddenPlasmaProperty):
    """
    Attributes
    ----------
    lines_lower_level_index : numpy.ndrarray, dtype int
        Levels data for lower levels of particular lines
    """

    outputs = ("lines_lower_level_index",)

    def calculate(self, levels, lines):
        levels_index = pd.Series(
            np.arange(len(levels), dtype=np.int64), index=levels
        )
        lines_index = lines.index.droplevel("level_number_upper")
        return np.array(levels_index.loc[lines_index])
```

On the other hand here's SpontRecombCoeff is in the continum_interactions_properties PlasmaPropertyCollection and it's evident from the code in the actual calculate method being highlighted as red that this class is never built from any of the plasma configs in test_complete_plasmas.

```python
class SpontRecombRateCoeff(ProcessingPlasmaProperty):
    """
    Attributes
    ----------
    alpha_sp : pandas.DataFrame, dtype float
        The rate coefficient for spontaneous recombination.
    """

    outputs = ("alpha_sp",)
    latex_name = (r"\alpha^{\textrm{sp}}",)

    def calculate(
        self,
        photo_ion_cross_sections,
        photo_ion_block_references,
        photo_ion_index,
        phi_ik,
        boltzmann_factor_photo_ion,
    ):
        cross_section = photo_ion_cross_sections["x_sect"].values
        nu = photo_ion_cross_sections["nu"].values

        alpha_sp = 8 * np.pi * cross_section * nu**2 / C**2
        alpha_sp = alpha_sp[:, np.newaxis]
        alpha_sp = alpha_sp * boltzmann_factor_photo_ion
        alpha_sp = integrate_array_by_blocks(
            alpha_sp, nu, photo_ion_block_references
        )
        alpha_sp = pd.DataFrame(alpha_sp, index=photo_ion_index)
        return alpha_sp * phi_ik.loc[alpha_sp.index]
```

Going through and seeing what is technically covered by test_complete_plasma is a good way to guide what's worth writing unit tests for, identifies features that are not used by the plasma module and should maybe be migrated to exclusively be used for type ii plasma, features that should be removed completely, or to find features that need to be fleshed out.

Plasma Graph Generation

As mentioned at the top of the document in the NetworkX section of the document, it's possible to generate diagrams of how the plasma graphs are connected together. Generating many of these diagrams with multiple configs is a good way to get a sense of what plasma properties are relevant to general tardis users.

This link is a github repo with many tardis configs: https://github.com/tardis-sn/tardis-configs

The link to documentation on how to draw the plasma graphs:
https://tardis-sn.github.io/tardis/how_to/output/how_to_plasma_graph.html

Potential GSOC 2027 Ideas/Tasks

- Create PRs with the uncovered and unreferenced code removed from the master branch and move this code into their own PRs
    - GSOC project would be dedicated to pushing the code into master in smaller chunks tested in their own PRs rather than pushing a bunch of untested code
    - Final project should have new tardis plasma properties implemented with example notebooks and example config files
    - The most feasible way to have a student refactor a new codebase
- Full refactor to use solvers instead of the NetworkC plasma
    - Working with mentors to flesh out the idea

I'm not sure how feasible these ideas are